UNIDAD 5
CONSULTAS

FICHA TÉCNICA DE LA UNIDAD DIDÁCTICA

Unidad Didáctica	Unidad Didáctica 5: Recuperación de Información en MySQL (Lenguaje de Manipulación de Datos - Consultas DML)
Resultados de Aprendizaje	**RA4:** Realiza consultas sobre los datos almacenados en la base de datos utilizando herramientas gráficas y el lenguaje de manipulación de datos.
Criterios de Evaluación	CE4.a, CE4.b, CE4.c, CE4.d, CE4.e, CE4.f, CE4.g, CE4.h, CE4.i
Tiempo Estimado	36 Horas (Planificado para 6 semanas de sesiones teórico-prácticas por su alta densidad de contenido)

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN AL LENGUAJE DML

2. CONSULTAS BÁSICAS SOBRE UNA TABLA

2.1 Estructura General

2.2 La Cláusula SELECT

2.2.1 Proyección Total (SELECT *)

2.2.2 Proyección Parcial de Columnas

2.2.3 Comentarios en Sentencias SQL

2.2.4 Expresiones y Columnas Calculadas

2.2.5 Renombrado de Columnas (AS)

2.2.6 Funciones Escalares (Cadenas, Matemáticas y Fechas)

2.2.7 Modificadores de Resultados: DISTINCT, ALL y DISTINCTROW

2.3 Cláusula WHERE (Filtrado)

2.3.1 Operadores de Comparación y Lógicos

2.3.2 Evaluación de Rangos (BETWEEN) y Listas de Valores (IN)

2.3.3 Búsqueda de Patrones de Texto (LIKE)

2.3.4 Tratamiento de la Ausencia de Datos (IS NULL / IS NOT NULL)

2.3.5 Filtrado por Tipos Especiales (Booleanos, Horas y Marcas de Tiempo)

2.4 Cláusula ORDER BY: Clasificación Física de Resultados

2.4.1 Ordenación Ascendente (ASC) y Descendente (DESC)

2.4.2 Ordenación por Múltiples Atributos

2.5 Cláusula LIMIT: Paginación y Control del Tránsito de Red

3. CONSULTAS SOBRE VARIAS TABLAS

3.1 Concepto de Producto Cartesiano (Composición Cruzada o CROSS JOIN)

3.2 Composición Interna (INNER JOIN)

3.2.1 Sintaxis Explícita (ON / USING) vs. Implícita (Equi-Join)

3.2.2 Uniones Equivalentes (Equi-Joins) y No Equivalentes (Non-Equi-Joins)

3.2.3 Autocomposición o Unión Reflexiva (Self-Join)

3.2.4 Composición de Tres o Más Tablas y Rutas de Intersección

3.3 Composición Externa (OUTER JOIN)

3.3.1 Composición Externa Izquierda (LEFT JOIN)

3.3.2 Composición Externa Derecha (RIGHT JOIN)

3.3.3 Ausencia de FULL OUTER JOIN en MySQL y su Simulación vía UNION

3.3.4 Composiciones Externas Naturales y sus Riesgos

3.4 Gestión de Tablas de Distintas Bases de Datos en el Mismo Servidor

4. CONSULTAS DE AGREGACIÓN Y RESUMEN

4.1 Funciones de
Agregación
4.1.1
Comportamiento y Diferencias entre COUNT(*) y COUNT(columna)
4.1.2
Agregación de Valores Únicos (COUNT DISTINCT)
4.1.3
Interacción de las Funciones de Agregación con Valores Nulos
4.2
Cláusula GROUP BY: Agrupamiento de Registros
4.2.1 Mecánica de
Particionado
4.2.2
Agrupamiento sobre Múltiples Atributos
4.3 Cláusula HAVING:
Filtrado de Grupos
4.3.1
Diferencias Críticas de Procesamiento: WHERE vs. HAVING
4.4
Errores Comunes en Consultas de Resumen y ONLY_FULL_GROUP_BY

5. SUBCONSULTAS (CONSULTAS ANIDADAS)

5.1
Introducción y Tipología de Subconsultas
5.2 Subconsultas de Valor
Escalar
5.3
Subconsultas de Fila o de Lista (Múltiples Filas)
5.3.1
Operadores de Comparación con Conjuntos: IN y NOT IN
5.3.2
Operadores de Cuantificación: ANY / SOME y ALL
5.3.3
El Peligro Crítico de NOT IN con Valores Nulos en la
Subconsulta
5.4
Subconsultas de Tabla (Cláusula FROM / Derived Tables)
5.5
Subconsultas Correlacionadas vs. No Correlacionadas
5.6
Comprobación de Existencia (EXISTS y NOT EXISTS)

5.7
Errores Comunes de Execution en Subconsultas

6. ERRORES COMUNES Y GUÍA DE BUENAS PRÁCTICAS

7. ANEXO I: FUNCIONES DE MYSQL MÁS UTILIZADAS

7.1 Funciones de Cadenas de Texto

7.2 Funciones Matemáticas

7.3 Funciones de Fecha y Hora

7.4 Funciones de Control de Flujo y Lógica

8. ANEXO II: SCRIPT SQL DE CREACIÓN Y CARGA PRÁCTICA (practica_dml)

1. INTRODUCCIÓN AL LENGUAJE DML

A diferencia de los lenguajes de programación imperativos tradicionales (C, Java, Python), donde el desarrollador define el algoritmo paso a paso (*cómo* obtener los datos), SQL es un lenguaje **declarativo**. El usuario describe formalmente la estructura de los datos que desea recuperar (*qué* obtener), y es el **Optimizador de Consultas** de MySQL el encargado de calcular el plan de ejecución físico más eficiente para acceder al disco y la memoria.

Para un Administrador de Sistemas, comprender el flujo interno de una consulta dentro de la instancia de MySQL es vital para diagnosticar latencias y cuellos de botella:

[Conexión Cliente] ---> [Hebra de Conexión] ---> [Analizador Sintáctico/Semántico (Parser)]
|
[Resultados al Cliente] <-- [Motor de Almacenamiento (InnoDB)] <-- [Optimizador de Consultas]

1. **Parser & Preprocesador:** Valida la sintaxis SQL de la consulta y verifica que las tablas y columnas existan en el diccionario de datos.
2. **Optimizador:** Evalúa múltiples planes de acceso alternativos (uso de índices primarios, secundarios, escaneo de tablas completas) y calcula el coste estimado de I/O de CPU y disco para elegir el plan óptimo.

3. **Motor de Ejecución (Storage Engine):** Solicita las páginas de datos requeridas al motor físico (generadamente InnoDB), aprovechando el Buffer Pool en memoria RAM para minimizar el acceso físico a disco.

Uno de los errores conceptuales más frecuentes en estudiantes de administración de sistemas es asumir que MySQL procesa las cláusulas en el mismo orden en que se escriben en el editor. El orden lógico de procesamiento determina de forma rigurosa la visibilidad de los alias de columnas y el alcance de los filtros:

```
Orden de Escritura
Orden de Ejecución Lógico


1. SELECT <columnas>
1. FROM <tablas> (Carga el origen de datos)


2. FROM <origen_datos>
2. ON <condiciones_unión> (Filtra las composiciones)


3. WHERE <condición_filtro>
3. JOIN <tablas_asociadas> (Instancia el producto
cartesiano)


4. GROUP BY <atributos>
4. WHERE <condiciones_tuplas> (Filtra filas individuales)


5. HAVING <condición_grupo>
5. GROUP BY <atributos> (Segmenta en subgrupos)


6. ORDER BY <atributos>
6. HAVING <condiciones_grupo> (Filtra los subgrupos
creados)


7. LIMIT <desplaz., cantidad>
7. SELECT <columnas> (Proyecta y calcula alias)
```

```
8. DISTINCT (Elimina duplicados remanentes)



9. ORDER BY <atributos> (Ordena el conjunto final de
salida)



10. LIMIT (Recorta la porción de datos solicitada)
```

```
Nota
de Diseño: Debido a que la cláusula SELECT se ejecuta en el
paso número , los alias definidos con la palabra clave AS no están
disponibles en la cláusula WHERE (paso ). No obstante, sí se pueden
emplear en ORDER BY (paso ) y LIMIT (paso ).
```

2. CONSULTAS BÁSICAS SOBRE UNA TABLA

2.1 Estructura General

La sintaxis mínima para invocar una proyección de datos se compone del comando SELECT acoplado al origen físico provisto por FROM.

🔗 **Documentación Oficial: Consulta la sintaxis gramatical formal y todos los modificadores disponibles para este comando de proyección en MySQL Manual - SELECT Statement.**

2.2 La Cláusula SELECT

2.2.1 Proyección Total (SELECT *)

Aunque el uso del asterisco comodín (*) es ampliamente recurrente en entornos de desarrollo rápido, **está estrictamente desaconsejado en entornos de producción** debido a:

1. **Saturación del Tránsito de Red:** Recuperar columnas innecesarias (especialmente campos TEXT o BLOB) desperdicia ancho de banda entre el servidor de base de datos y el servidor de aplicaciones.

2. **Invalidación de Índices de Cubrimiento (Covering Indexes):** Si una consulta requiere solo dos columnas indexadas, y se utiliza *, MySQL se verá obligado a leer los bloques de datos de la clave primaria del disco (o el índice agrupado) en lugar de servir la petición utilizando únicamente el índice secundario en memoria RAM.

2.2.2 Proyección Parcial de Columnas

Especifica explícitamente los campos requeridos, optimizando el rendimiento físico de la consulta.

```
-- Recuperación optimizada de datos identificativos de empleados
SELECT codigo_empleado, nombre, apellido1, email
FROM empleado;
```

2.2.3 Comentarios en Sentencias SQL

```
-- Comentario de línea única estándar SQL
# Comentario de línea única específico de MySQL
/*
Comentario multilinea
de utilidad para documentar scripts SysAdmin complejos
*/
```

2.2.4 Expresiones y Columnas Calculadas

La cláusula SELECT puede computar expresiones matemáticas directas de tipo constante o valores dinámicos derivados de los atributos de cada registro.

```
-- Cálculo dinámico de precios incrementados con el IVA (21%)
SELECT codigo_producto, nombre, precio_venta, (precio_venta *
1.21)
FROM producto;
```

2.2.5 Renombrado de Columnas (AS)

Para dotar de legibilidad semántica a las columnas calculadas, se asignan alias mediante la cláusula AS.

```
SELECT codigo_producto, nombre, precio_venta AS precio_base,
(precio_venta * 1.21) AS precio_pvd_iva
FROM producto;
```

2.2.6 Funciones Escalares (Cadenas, Matemáticas y Fechas)

MySQL implementa un vasto catálogo de funciones de transformación rápida en el hilo de ejecución de la consulta.

```
-- Manipulación avanzada de cadenas de texto y fechas de pedidos
SELECT
CONCAT(nombre, ' ', apellido1) AS nombre_completo,
UPPER(email) AS email_mayusculas
FROM empleado;
SELECT
fecha_esperada,
fecha_entrega,
DATEDIFF(fecha_entrega, fecha_esperada) AS desviacion_dias
FROM pedido;
```

2.2.7 Modificadores de Resultados: DISTINCT, ALL y DISTINCTROW

El modificador DISTINCT instruye al motor a ordenar temporalmente los resultados de salida para purgar todas aquellas tuplas duplicadas.

```
-- Recuperación de los códigos de fabricantes únicos que tienen
productos registrados
SELECT DISTINCT codigo_fabricante
FROM producto;
```

2.3 Cláusula WHERE (Filtrado)

La cláusula WHERE evalúa condiciones lógicas sobre las filas del origen de datos. Solo aquellas filas cuya evaluación resulta en TRUE pasan al siguiente paso del flujo lógico.

2.3.1 Operadores de Comparación y Lógicos

- **Comparadores:** =, <>, !=, <, <=, >, >=.

- **Conectores lógicos:** AND, OR, NOT.

```
-- Filtrado estricto combinado
SELECT nombre, gama, precio_venta
FROM producto
WHERE gama = 'Herramientas' AND precio_venta >= 100.00;
```

2.3.2 Evaluación de Rangos (BETWEEN) y Listas de Valores (IN)

```
-- Filtrado por rango inclusivo (Equivale a precio_venta >= 50
AND precio_venta <= 150)
SELECT nombre, precio_venta
FROM producto
WHERE precio_venta BETWEEN 50.00 AND 150.00;
-- Filtrado por pertenencia a una lista discreta de valores
SELECT nombre, gama
FROM producto
WHERE gama IN ('Herramientas', 'Frutales', 'Ornamentales');
```

2.3.3 Búsqueda de Patrones de Texto (LIKE)

Permite la comparación parcial de strings mediante caracteres comodín:

- % representa cualquier secuencia de cero o más caracteres.
- _ representa exactamente un único carácter físico.

```
-- Recupera productos cuyo nombre empiece con "Maza", contenga
cualquier carácter intermedio y termine en "a"
SELECT nombre
FROM producto
WHERE nombre LIKE 'Maza%a';
-- Recupera correos que tengan exactamente un carácter antes de la
arroba (Ej. "j@empresa.com")
SELECT email
FROM empleado
WHERE email LIKE '_@%';
```

2.3.4 Tratamiento de la Ausencia de Datos (IS NULL / IS NOT NULL)

En el modelo relacional y SQL, NULL no equivale a un valor numérico cero, ni a una cadena de texto vacía "". Representa la "ausencia de valor" o "desconocimiento". Las comparaciones directas con = (como columna = NULL) **siempre devolverán un resultado de tipo indeterminado (UNKNOWN)** bajo la lógica triestatal de SQL. Por ende, para evaluar la presencia o ausencia de datos, deben emplearse de forma exclusiva los operadores estructurados IS NULL y IS NOT NULL.

```
-- IDENTIFICACIÓN CORRECTA de clientes que no tienen representante
de ventas asociado
SELECT nombre_cliente
FROM cliente
WHERE codigo_empleado_rep_ventas IS NULL;
```

> 🔗 Documentación Oficial: Para comprender a fondo el funcionamiento de la lógica de tres valores (True, False, Unknown) y el manejo de marcas de ausencia, lee MySQL Manual - Working with NULL Values.

2.3.5 Filtrado especiales (Booleanos, Horas y Marcas de Tiempo)

Para un administrador de sistemas, es fundamental controlar el filtrado de estados booleanos y rangos temporales específicos de tipo TIME o DATETIME:

```
-- 1. Filtrado de campos BOOLEAN (Soportados internamente como
TINYINT(1) en MySQL)
-- Recupera únicamente los clientes VIP que tienen activado el estado
premium
SELECT nombre_cliente, pais, limite_credito
FROM cliente
WHERE es_premium = TRUE; -- También se puede escribir simplemente:
WHERE es_premium
-- 2. Filtrado por rango horario específico (tipo TIME)
-- Recupera los empleados cuyo turno de mañana inicia a las 08:00:00
o antes
SELECT nombre, apellido1, hora_entrada
FROM empleado
WHERE hora_entrada <= '08:00:00';
-- 3. Filtrado por marcas de tiempo completas (tipo DATETIME o
TIMESTAMP)
-- Recupera los clientes que se registraron a partir de una fecha y
hora específicas en producción
SELECT nombre_cliente, fecha_alta
```

```
FROM cliente
WHERE fecha_alta >= '2026-01-01 00:00:00';
```

2.4 Cláusula ORDER BY: Clasificación Física de Resultados

Ordena el conjunto final de registros devueltos. Por defecto, si no se especifica el sentido, el motor aplica un orden ascendente (ASC).

2.4.1 Ordenación Ascendente (ASC) y Descendente (DESC)

```
-- Ordenación explícita descendente de precios
SELECT nombre, precio_venta
FROM producto
WHERE gama = 'Herramientas'
ORDER BY precio_venta DESC;
```

2.4.2 Ordenación por Múltiples Atributos

Permite la ordenación jerárquica: si hay valores idénticos en la primera columna, se clasifican según los criterios de las columnas sucesivas.

```
-- Ordena alfabéticamente por apellido; ante apellidos iguales,
ordena de forma descendente por nombre
SELECT apellido1, nombre
FROM empleado
ORDER BY apellido1 ASC, nombre DESC;
```

2.5 Cláusula LIMIT: Paginación y Control del Tránsito de Red

La cláusula LIMIT restringe la cantidad de tuplas enviadas de vuelta al host cliente, lo cual es vital para estructurar consultas de paginación en interfaces y limitar el consumo de RAM de los hilos de conexión.

```
-- Recuperación de los 5 productos más caros de la tienda
SELECT nombre, precio_venta
FROM producto
```

```
ORDER BY precio_venta DESC
LIMIT 5;
-- Paginación avanzada: LIMIT <desplazamiento (offset)>,
<cantidad_tuplas>
-- Salta los primeros 10 registros y recupera las 5 filas siguientes
(Registros del 11 al 15)
SELECT nombre, precio_venta
FROM producto
ORDER BY precio_venta DESC
LIMIT 10, 5;
```

🔗 Documentación Oficial: Revisa detalles técnicos y de optimización de la cláusula de limitación física de filas en MySQL Manual - LIMIT Query Optimization.

3. CONSULTAS SOBRE VARIAS TABLAS

3.1 Concepto de Producto Cartesiano (Composición Cruzada o CROSS JOIN)

Si intentamos consultar múltiples tablas en una sentencia separándolas por comas en el FROM, o invocando explícitamente CROSS JOIN sin especificar una condición de enlace, el motor ejecutará un **producto cartesiano**. Esto significa que asociará cada registro de la primera tabla con absolutamente todos los registros de la segunda tabla.

Filas resultantes= Filas de Tabla A x Filas de Tabla B

Si la Tabla A contiene 10000 registros y la Tabla B 5000, el producto cartesiano instanciará temporalmente en el servidor un conjunto de **50 millones de registros**, saturando la memoria del proceso mysqld y pudiendo bloquear el servicio.

```
-- Producto cartesiano explícito (Ejecutar solo bajo estricta
necesidad de diseño)
SELECT * FROM fabricante CROSS JOIN producto;
```

3.2 Composición Interna (INNER JOIN)

La composición interna evalúa la intersección estricta de dos o más conjuntos. El motor devolverá únicamente aquellas tuplas que tengan una coincidencia real en ambas tablas basándose en la condición lógica de emparejamiento.

3.2.1 Sintaxis Explícita (ON / USING) vs. Implícita (Equi-Join)

Existen dos vías para implementar la unión física, siendo la explícita mediante la cláusula INNER JOIN ... ON la estándar y recomendada para preservar la legibilidad del código frente a filtros lógicos.

```
-- VÍA 1: Sintaxis Explícita Estándar (Recomendada)
SELECT producto.nombre, fabricante.nombre AS fabricante
FROM producto
INNER JOIN fabricante
ON producto.codigo_fabricante = fabricante.codigo;
-- VÍA 2: Sintaxis Explícita abreviada mediante USING (Válida solo si
ambas columnas se llaman idéntico en las dos tablas)
-- SELECT producto.nombre, fabricante.nombre FROM producto INNER JOIN
fabricante USING (codigo_fabricante);
-- VÍA 3: Sintaxis Implícita Clásica (Desaconsejada por mezclar
filtros relacionales con condiciones lógicas)
SELECT producto.nombre, fabricante.nombre AS fabricante
FROM producto, fabricante
WHERE producto.codigo_fabricante = fabricante.codigo;
```

🔗 Documentación Oficial: Puedes aprender más sobre los algoritmos internos de MySQL para resolver uniones, como el Nested-Loop Join, en MySQL Manual - JOIN Syntax.

3.2.2 Uniones Equivalentes y No Equivalentes

- **Equi-Join:** La condición de enlace utiliza estrictamente el operador de igualdad (=). Es la más común para enlazar Claves Ajenas con Claves Primarias.
- **Non-Equi-Join:** La condición de enlace utiliza operadores relacionales diferentes al de igualdad (<, >, BETWEEN, etc.).

```
-- Ejemplo de Non-Equi-Join para asociar salarios con categorías de
rango de nómina
SELECT E.nombre, E.salario, G.grado_salarial
FROM empleado E
INNER JOIN grado_nomina G
ON E.salario BETWEEN G.salario_minimo AND G.salario_maximo;
```

3.2.3 Autocomposición o Unión Reflexiva (Self-Join)

La autocomposición consiste en relacionar una tabla consigo misma. Para implementarla, **es obligatorio el uso de alias de tabla** para simular ante el analizador de MySQL que se trata de dos orígenes de datos físicos distintos.

```
-- Recupera el nombre de los empleados junto al nombre de su
supervisor directo
SELECT
E.nombre AS empleado,
J.nombre AS jefe
FROM empleado E
INNER JOIN empleado J
ON E.codigo_jefe = J.codigo_empleado;
```

3.2.4 Composición de Tres o Más Tablas y Rutas de Intersección

Al unir más de dos tablas, el orden semántico debe definir un camino relacional coherente para evitar la aparición accidental de productos cartesianos intermedios.

```
-- Unión encadenada: Cliente -> Empleado (Rep Ventas) ->
Oficina
SELECT
C.nombre_cliente,
CONCAT(E.nombre, ' ', E.apellido1) AS representante,
O.ciudad AS ciudad_oficina
FROM cliente C
INNER JOIN empleado E ON C.codigo_empleado_rep_ventas =
E.codigo_empleado
INNER JOIN oficina O ON E.codigo_oficina = O.codigo_oficina;
```

3.3 Composición Externa (OUTER JOIN)

En ocasiones, requerimos recuperar los registros de una tabla aunque no tengan correspondencia o relación en la otra tabla (por ejemplo, queremos listar todos los fabricantes, tengan o no productos en nuestro catálogo). Para este fin se utilizan las composiciones externas.

3.3.1 Composición Externa Izquierda (LEFT JOIN)

Recupera absolutamente **todas las filas de la tabla declarada a la izquierda** de la instrucción LEFT JOIN. Para aquellos registros que no posean una correspondencia en la tabla de la derecha, el motor rellenará los atributos correspondientes a esta última con valores NULL.

```
-- Recupera TODOS los fabricantes de la base de datos con sus
respectivos productos
-- Fabricantes como "Xiaomi" que no tienen productos, aparecerán con
NULL en las columnas de producto
SELECT fabricante.nombre, producto.nombre AS producto
FROM fabricante
LEFT JOIN producto
ON fabricante.codigo = producto.codigo_fabricante;
```

3.3.2 Composición Externa Derecha (RIGHT JOIN)

Funciona de forma análoga a LEFT JOIN pero preservando la totalidad del conjunto de datos de la **tabla declarada a la derecha** de la instrucción.

```
-- Comportamiento equivalente al anterior invirtiendo el orden de
las tablas
SELECT fabricante.nombre, producto.nombre AS producto
FROM producto
RIGHT JOIN fabricante
ON producto.codigo_fabricante = fabricante.codigo;
```

Recomendación de Ingeniería de Software: Se prefiere el uso sistemático de LEFT JOIN por legibilidad y facilidad de lectura humana (procesamiento de izquierda a derecha), evitando la alternancia confusa con RIGHT JOIN en el mismo bloque de código.

3.3.3 Ausencia de FULL OUTER JOIN en MySQL

El estándar SQL define la sentencia FULL OUTER JOIN para recuperar la unión externa de ambos extremos (coincidentes y no coincidentes). MySQL **no implementa de forma nativa** esta composición. Sin embargo, los administradores pueden simular este comportamiento exacto ejecutando un LEFT JOIN y un RIGHT JOIN unificados mediante el operador de conjunto UNION (el cual remueve duplicados del set final).

```
-- Simulación de FULL OUTER JOIN en MySQL
SELECT E.nombre AS empleado, D.nombre AS departamento
FROM empleado E
LEFT JOIN departamento D ON E.codigo_departamento = D.codigo
UNION
SELECT E.nombre AS empleado, D.nombre AS departamento
FROM empleado E
RIGHT JOIN departamento D ON E.codigo_departamento = D.codigo;
```

3.3.4 Composiciones Externas Naturales y sus Riesgos

La instrucción NATURAL LEFT JOIN o NATURAL RIGHT JOIN asocia las tablas buscando implícitamente columnas con nombres exactamente idénticos en ambos extremos. **Su uso en entornos corporativos está estrictamente prohibido** debido a que un cambio de estructura físico posterior en el DDL (por ejemplo, añadir una columna de auditoría llamada fecha_modificacion en ambas tablas) alterará de forma invisible e instantánea la condición de enlace del JOIN, rompiendo la lógica de la aplicación.

3.4 Gestión de Tablas de Distintas Bases de Datos en el Mismo Servidor

Si el usuario de la conexión de base de datos posee los privilegios globales necesarios (SELECT_PRIV), MySQL permite enlazar datos entre esquemas distintos dentro de la misma instancia cualificando los objetos mediante el prefijo del nombre del esquema: <nombre_bd>.<nombre_tabla>.

```
-- Intersección entre bases de datos de entornos de facturación e
inventario (Ejemplo ilustrativo de producción)
SELECT F.id_factura, I.nombre_articulo
FROM facturacion_prod.facturas F
INNER JOIN inventario_prod.articulos I
ON F.codigo_articulo = I.id_articulo;
```

4. CONSULTAS DE AGREGACIÓN Y RESUMEN

Las consultas de resumen reducen la granularidad de los datos individuales, transformando conjuntos masivos de filas en tuplas analíticas consolidadas.

4.1 Funciones de Agregación

Las funciones de agregación operan sobre colecciones de valores de una columna y devuelven un único valor escalar descriptivo del conjunto.

4.1.1 Comportamiento y Diferencias entre COUNT(*) y COUNT(columna)

- COUNT(*): Cuenta el número absoluto de filas físicas que componen el conjunto de resultados, **incluyendo tuplas con valores nulos o duplicados**.
- COUNT(columna): Evalúa la columna especificada y devuelve el número de registros en los que dicha columna posee un valor **distinto de NULL**.

```
-- Caso de estudio en producción (basado en nuestra tabla
'cliente'):
-- Supongamos clientes con y sin representante de ventas asignado
SELECT COUNT(*) FROM cliente; -- Resultado: Conteo total de
clientes
SELECT COUNT(codigo_empleado_rep_ventas) FROM cliente; -- Cuenta solo
los que no tengan NULL en esa columna
```

4.1.2 Agregación de Valores Únicos (COUNT DISTINCT)

Permite omitir los valores duplicados antes de ejecutar la acción aritmética de conteo de registros.

```
-- Determina cuántos países distintos tienen clientes
registrados
SELECT COUNT(DISTINCT pais) FROM cliente;
```

4.1.3 Interacción de las Funciones de Agregación con Valores Nulos

Con la excepción de COUNT(*), todas las funciones de agregación (SUM, AVG, MIN, MAX) **ignoran por completo los valores NULL** para sus cálculos matemáticos:

```
-- AVG calcula la media dividiendo la suma total únicamente entre
los registros que NO tengan NULL
-- En nuestro script de prácticas, 'comision' puede tener valores
```

```
NULL
SELECT AVG(comision) FROM empleado;
```

4.2 Cláusula GROUP BY: Agrupamiento de Registros

La cláusula GROUP BY subdivide el conjunto de datos de la tabla original en pequeños subgrupos lógicos basados en el valor común de una o más columnas especificadas.

4.2.1 Mecánica de Particionado

Datos de entrada (Tabla Producto):
[Gama: Htas, Precio: 10]
[Gama: Htas, Precio: 20] GROUP BY Gama [Gama: Htas, Suma: 30]
[Gama: Jrdn, Precio: 15] ----------> [Gama: Jrdn, Suma: 15]

```
-- Suma total de los precios de venta agrupados por el código del
fabricante asociado
SELECT codigo_fabricante, SUM(precio_venta) AS total_ventas
FROM producto
GROUP BY codigo_fabricante;
```

4.2.2 Agrupamiento sobre Múltiples Atributos

Permite la creación de subgrupos anidados para informes detallados multi-nivel.

```
-- Agrupamiento cruzado por gama y fabricante
SELECT gama, codigo_fabricante, COUNT(*) AS total_productos
FROM producto
GROUP BY gama, codigo_fabricante;
```

4.3 Cláusula HAVING: Filtrado de Grupos

La cláusula HAVING aplica filtros condicionales a los registros consolidados **después de que se ha ejecutado el agrupamiento físico** de los datos.

4.3.1 Diferencias Críticas de Procesamiento: WHERE vs. HAVING

- WHERE: Se procesa **antes** de agrupar los datos (GROUP BY). Filtra registros individuales de la tabla. **Nunca** puede contener una función de agregación (por ejemplo,

WHERE SUM(precio_venta) > 100 lanzará un error de sintaxis).

- HAVING: Se procesa **después** de agrupar los datos. Filtra los resultados consolidados de las funciones de agregación basándose en el grupo completo.

```
-- FILTRADO COMBINADO CORRECTO:
-- Filtra primero los productos que valen más de 10 euros y, tras
agrupar por fabricante,
-- muestra solo aquellos fabricantes que tienen más de 1 producto que
cumpla dicho requisito.
SELECT codigo_fabricante, COUNT(*) AS volumen_productos
FROM producto
WHERE precio_venta > 10.00
GROUP BY codigo_fabricante
HAVING COUNT(*) > 1;
```


```
-- FILTRADO COMBINADO CORRECTO:
-- Filtra primero los productos que valen más de 10 euros y, tras
agrupar por fabricante,
-- muestra solo aquellos fabricantes que tienen más de 1 producto que
cumpla dicho requisito.
SELECT codigo_fabricante, COUNT(*) AS volumen_productos
FROM producto
WHERE precio_venta > 10.00
GROUP BY codigo_fabricante
HAVING COUNT(*) > 1;
```


4.4 Errores Comunes en Consultas de Resumen y ONLY_FULL_GROUP_BY

Uno de los mayores dolores de cabeza para los administradores de sistemas al desplegar código heredado en MySQL 8.0+ es el comportamiento derivado del modo estricto de SQL ONLY_FULL_GROUP_BY.

```
-- CONSULTA INVÁLIDA bajo el estándar de producción (Lanza ERROR
1055)
SELECT codigo_fabricante, nombre, COUNT(*)
FROM producto
GROUP BY codigo_fabricante;
```

```
-- CONSULTA INVÁLIDA bajo el estándar de producción (Lanza ERROR
1055)
SELECT codigo_fabricante, nombre, COUNT(*)
FROM producto
GROUP BY codigo_fabricante;
```

Explicación Técnica del Error: En la consulta anterior, agrupamos los datos por codigo_fabricante. Sin embargo, en el SELECT estamos solicitando la proyección de la columna nombre. Como pueden existir muchos nombres de productos distintos para un mismo código de fabricante, el motor no sabe qué nombre de producto individual mostrar en la tupla de salida. Para solucionar esto sin desactivar la variable global sql_mode, la consulta debe reescribirse para que todas las columnas proyectadas en el SELECT estén presentes en el GROUP BY o se encuentren dentro de una función de agregación (como MAX(nombre) o GROUP_CONCAT(nombre)).

🔗 **Documentación Oficial:** Amplía información sobre el control de agregaciones de tipo unívoco y las políticas de restricción de SQL en MySQL Manual - GROUP BY Handling and ONLY_FULL_GROUP_BY.

5. SUBCONSULTAS (CONSULTAS ANIDADAS)

Una subconsulta es una sentencia SELECT anidada dentro de otra consulta de nivel superior (llamada consulta externa). Las subconsultas permiten estructurar soluciones complejas sin requerir múltiples pasos o llamadas desde el lenguaje de programación de backend.

5.1 Introducción y Tipología de Subconsultas

Según el resultado que devuelve la consulta interna, las clasificamos en:

1. **Escalar:** Devuelve una única fila con una única columna (un solo valor).
2. **De Fila:** Devuelve una única fila con múltiples columnas asociadas.
3. **De Lista / Columna:** Devuelve una columna con múltiples registros lógicos.
4. **De Tabla:** Devuelve un conjunto bidimensional completo de filas y columnas (Derived Table).

🔗 **Documentación Oficial:** Consulta las reglas de análisis semántico del anidamiento de sentencias DML en MySQL Manual - Subquery Syntax.

5.2 Subconsultas de Valor Escalar

Suelen acoplarse con operadores de comparación relacionales clásicos en las cláusulas WHERE o HAVING.

```
-- Obtener los datos de los productos que tienen el precio máximo
registrado
SELECT * FROM producto
WHERE precio_venta = (SELECT MAX(precio_venta) FROM producto);
```


```
-- Obtener los datos de los productos que tienen el precio máximo
registrado
SELECT * FROM producto
WHERE precio_venta = (SELECT MAX(precio_venta) FROM producto);
```


5.3 Subconsultas de Fila o de Lista (Múltiples Filas)

5.3.1 Operadores de Comparación con Conjuntos: IN y NOT IN

- IN: Evalúa si el atributo de la consulta externa coincide con alguno de los elementos del conjunto devuelto por la subconsulta.
- NOT IN: Evalúa que el atributo de la consulta externa no coincida con ningún elemento del conjunto devuelto por la subconsulta.

```
-- Obtener productos de fabricantes ubicados en España
SELECT * FROM producto
WHERE codigo_fabricante IN (SELECT codigo FROM fabricante WHERE pais
= 'España');
```


```
-- Obtener productos de fabricantes ubicados en España
SELECT * FROM producto
WHERE codigo_fabricante IN (SELECT codigo FROM fabricante WHERE pais
= 'España');
```

5.3.2 Operadores de Cuantificación: ANY / SOME y ALL

- ANY o SOME: El criterio de comparación se cumple si la evaluación lógica resulta en TRUE para **al menos uno** de los valores del conjunto de la subconsulta (Equivale semánticamente a un conector relacional OR masivo).
- ALL: El criterio de comparación se cumple si la evaluación lógica resulta en TRUE para **absolutamente todos** los elementos del conjunto devuelto por la subconsulta (Equivale a un conector relacional AND masivo).

```
-- Obtener los productos cuyo precio de venta sea mayor que TODOS
los productos de la gama 'Herramientas'
SELECT nombre, precio_venta
FROM producto
WHERE precio_venta > ALL (SELECT precio_venta FROM producto WHERE
gama = 'Herramientas');
```


```
-- Obtener los productos cuyo precio de venta sea mayor que TODOS
los productos de la gama 'Herramientas'
SELECT nombre, precio_venta
FROM producto
WHERE precio_venta > ALL (SELECT precio_venta FROM producto WHERE
gama = 'Herramientas');
```


5.3.3 El Peligro Crítico de NOT IN con Valores Nulos en la Subconsulta

Advertencia: Si la subconsulta interna invocada mediante un operador NOT IN devuelve, en su conjunto de datos, al menos un solo registro con valor NULL, **la consulta externa completa no devolverá ninguna fila**.

Esto se debe a que SQL evalúa la condición de exclusión como:

$$\text{valor} \neq \text{item}_1 \text{ AND valor} \neq \text{item}_2 \text{ AND valor} \neq \text{NULL}$$

Cualquier comparación de desigualdad física con NULL resulta en UNKNOWN, invalidando toda la expresión lógica conectorizada con AND. Para mitigar esto, es obligatorio filtrar explícitamente los valores nulos en el filtro de la subconsulta interna:

```
-- CONTROL CORRECTO DE NOT IN PARA SEGURIDAD DE DATOS
SELECT nombre
FROM departamento
WHERE codigo NOT IN (
SELECT codigo_departamento
FROM empleado
WHERE codigo_departamento IS NOT NULL
);
```


```
-- CONTROL CORRECTO DE NOT IN PARA SEGURIDAD DE DATOS
SELECT nombre
FROM departamento
WHERE codigo NOT IN (
SELECT codigo_departamento
FROM empleado
WHERE codigo_departamento IS NOT NULL
);
```


5.4 Subconsultas de Tabla (Cláusula FROM / Derived Tables)

Cuando se anida una subconsulta dentro de la cláusula FROM, esta actúa temporalmente como una tabla lógica residente en memoria (tabla derivada o inline view). **En MySQL, es estrictamente obligatorio asignar un alias de tabla a estas estructuras** para evitar fallos sintácticos graves del parser del sistema.

```
-- Uso de Derived Table con alias 'subtabla'
SELECT AVG(subtabla.precio_maximo) AS media_maxima
FROM (
SELECT codigo_fabricante, MAX(precio_venta) AS precio_maximo
FROM producto
GROUP BY codigo_fabricante
) AS subtabla;
```


```
-- Uso de Derived Table con alias 'subtabla'
SELECT AVG(subtabla.precio_maximo) AS media_maxima
FROM (
SELECT codigo_fabricante, MAX(precio_venta) AS precio_maximo
FROM producto
```

```
GROUP BY codigo_fabricante
) AS subtabla;
```

5.5 Subconsultas Correlacionadas vs. No Correlacionadas

- **Subconsulta No Correlacionada:** Se ejecuta una única vez. El motor ejecuta la consulta interna de forma autónoma y proporciona el resultado final a la consulta externa.
- **Subconsulta Correlacionada:** El filtro de la consulta interna **hace referencia explícita a una columna de la tabla declarada en la consulta externa**. En consecuencia, el optimizador debe evaluar la subconsulta de forma interactiva **para cada una de las filas candidatas** de la consulta externa, lo cual puede degradar gravemente el rendimiento en tablas con millones de registros si no existen índices optimizados.

```
-- Ejemplo correlacionado: Encuentra productos que son más caros que
la media de su propia gama
SELECT P_Ext.nombre, P_Ext.precio_venta, P_Ext.gama
FROM producto P_Ext
WHERE P_Ext.precio_venta > (
SELECT AVG(P_Int.precio_venta)
FROM producto P_Int
WHERE P_Int.gama = P_Ext.gama
);
```

```
-- Ejemplo correlacionado: Encuentra productos que son más caros que
la media de su propia gama
SELECT P_Ext.nombre, P_Ext.precio_venta, P_Ext.gama
FROM producto P_Ext
WHERE P_Ext.precio_venta > (
SELECT AVG(P_Int.precio_venta)
FROM producto P_Int
WHERE P_Int.gama = P_Ext.gama
);
```

🔗 **Documentación Oficial:** Para comprender cómo el motor reescribe u optimiza de manera interna las subconsultas correlacionadas, consulta MySQL Manual - Optimizing Subqueries.

5.6 Comprobación de Existencia (EXISTS y NOT EXISTS)

El operador EXISTS devuelve TRUE si la subconsulta interna encuentra al menos un registro que cumpla la condición. Es un operador altamente optimizado por MySQL: una vez que el motor lo localiza, interrumpe inmediatamente el escaneo de la subconsulta (early exit), minimizando las operaciones de lectura en disco.

```
-- Obtener aquellos clientes que han efectuado al menos un
pedido
SELECT C.nombre_cliente
FROM cliente C
WHERE EXISTS (
SELECT 1
FROM pedido P
WHERE P.codigo_cliente = C.codigo_cliente
);
```


```
-- Obtener aquellos clientes que han efectuado al menos un
pedido
SELECT C.nombre_cliente
FROM cliente C
WHERE EXISTS (
SELECT 1
FROM pedido P
WHERE P.codigo_cliente = C.codigo_cliente
);
```


Rendimiento: En la subconsulta interna de un EXISTS o NOT EXISTS, es irrelevante la columna proyectada (SELECT *, SELECT 1, etc.). Se aconseja utilizar SELECT 1 para facilitar la lectura del código, ya que el optimizador descarta la proyección de columnas y se centra en validar la condición del filtro de correlación.

5.7 Errores Comunes de Execution en Subconsultas

Error 1242 (Subquery returns more than 1 row)

Ocurre cuando una subconsulta asociada a un comparador unitario (como =, < o >) devuelve dos o más registros en tiempo de ejecución de la base de datos.

```
-- ERROR EN PRODUCCIÓN si se intenta buscar un precio máximo con
operadores de igualdad y la subconsulta devuelve varios valores
```

```
-- SELECT * FROM producto WHERE precio_venta = (SELECT precio_venta
FROM producto ORDER BY precio_venta DESC LIMIT 2);
```

```
-- ERROR EN PRODUCCIÓN si se intenta buscar un precio máximo con
operadores de igualdad y la subconsulta devuelve varios valores
-- SELECT * FROM producto WHERE precio_venta = (SELECT precio_venta
FROM producto ORDER BY precio_venta DESC LIMIT 2);
```

Para solucionarlo, debemos modificar el comparador = por un operador de conjunto como IN o redefinir el límite de la subconsulta para asegurar la obtención de un único valor escalar (ej. mediante MAX()).

6. ERRORES COMUNES Y GUÍA DE BUENAS PRÁCTICAS

Para mantener un servidor MySQL con altas prestaciones bajo cargas de trabajo corporativas, el Administrador de Sistemas debe vigilar el comportamiento del DML:

1. **Evitar las composiciones cruzadas accidentales (CROSS JOIN implícitos):** Validar siempre que cada INNER JOIN o LEFT JOIN cuente con su correspondiente cláusula restrictiva de coincidencia física (ON o USING).

2. ```
   Utilizar filtros WHERE sobre columnas indexadas:
   Para evitar lecturas completas de tabla (Full Table Scans), asegúrate de
   que los filtros de búsqueda frecuente y las columnas que enlazan JOINs
   cuenten con un índice secundario asociado en la estructura DDL de la
   base de datos.
   ```

3. ```
   No utilizar funciones directamente sobre columnas
   filtradas: Si consultas WHERE YEAR(fecha_registro) = 2026,
   MySQL se verá obligado a procesar la función matemática para cada fila
   de la tabla, invalidando los índices asociados a fecha_registro. En su
   lugar, utiliza rangos planos: WHERE fecha_registro BETWEEN '2026-01-01'
   AND '2026-12-31'.
   ```

4. **Validar las consultas analíticas en producción:** Herramientas como EXPLAIN (analizadas a fondo en la Unidad 10) deben ser ejecutadas de forma sistemática ante

cualquier consulta que devuelva latencias de respuesta superiores a los 100 ms en los logs de consultas lentas (slow_query_log).

7. ANEXO I: FUNCIONES DE MYSQL MÁS UTILIZADAS

Este catálogo técnico detalla el comportamiento de las funciones nativas indispensables para la transformación de datos directamente en el motor de bases de datos.

🔗 **Documentación Oficial:** Tienes a tu disposición la guía de referencia alfabética y funcional de operadores y funciones nativas de MySQL en [MySQL Manual - Functions and Operators].

7.1 Funciones de Cadenas de Texto

-

```
SELECT CONCAT(nombre, ' ', apellido1) AS nombre_completo FROM
empleado;
SELECT UPPER(email) FROM empleado;
SELECT SUBSTRING(nombre, 1, 3) AS iniciales FROM empleado;
SELECT nombre, CHAR_LENGTH(nombre) AS longitud FROM producto;
SELECT REPLACE(email, 'empresa.com', 'corporativo.es') FROM
empleado;
SELECT TRIM(' Texto con espacios ') AS limpio;
```

🔗 **Documentación Oficial:** Para explorar todas las operaciones de procesamiento de texto con codificación multibyte (utf8mb4), accede a [MySQL Manual - String Functions].

7.2 Funciones Matemáticas

-

```
SELECT ROUND(precio_venta, 1) FROM producto;
SELECT FLOOR(15.99) AS bajo, CEIL(15.01) AS alto;
SELECT ABS(-25.35) AS valor_absoluto;
SELECT MOD(10, 3) AS resto;
SELECT POW(2, 3) AS resultado_potencia;
```

🔗 **Documentación Oficial:** El desglose completo de constantes y operaciones aritméticas de precisión de punto flotante se encuentra en MySQL Manual - Numeric Functions.

7.3 Funciones de Fecha y Hora

-

```
SELECT NOW() AS registro_temporal_servidor;
SELECT CURDATE() AS fecha_hoy;
SELECT DATE_ADD(CURDATE(), INTERVAL 30 DAY) AS fecha_vencimiento;
SELECT DATEDIFF(fecha_entrega, fecha_esperada) FROM pedido;
SELECT DATE_FORMAT(fecha_esperada, '%d/%m/%Y') AS formato_espanol
FROM pedido;
SELECT YEAR(fecha_esperada) AS anio_pedido FROM pedido;
```

🔗 **Documentación Oficial:** El tratamiento de zonas horarias e intervalos temporales complejos está descrito de forma exhaustiva en MySQL Manual - Date and Time Functions.

7.4 Funciones de Control de Flujo y Lógica

-

```
SELECT COALESCE(comision, 0.00) AS comision_final FROM empleado;
SELECT IFNULL(comision, 0) FROM empleado;
SELECT nombre, IF(precio_venta > 100, 'Caro', 'Asequible') AS
categoria FROM producto;
SELECT nombre, CASE WHEN precio_venta < 50 THEN 'Bajo' WHEN
precio_venta BETWEEN 50 AND 150 THEN 'Medio' ELSE 'Alto' END AS
rango_precio FROM producto;
```

🔗 **Documentación Oficial:** Descubre más sobre la evaluación perezosa y estructuras condicionales aplicadas al conjunto de datos en MySQL Manual - Flow Control Functions.

8. ANEXO II: SCRIPT SQL

Este script DDL y DML autoejecutable genera de forma íntegra el esquema físico para que puedan probar la totalidad de consultas y funciones explicadas en el tema.

🔗 **Documentación Oficial:** Para comprender el comportamiento transaccional del motor utilizado en este script, la gestión del almacenamiento en espacio de tablas (*tablespaces*) y

las políticas de bloqueo físico, puedes visitar MySQL Manual - InnoDB Storage Engine.

```
--
================================================================================
-- BASE DE DATOS DE PRÁCTICAS DML - UNIDAD 5 (ESQUEMA ENRIQUECIDO
PARA ASIR)
-- Diseñado para el ciclo de ASIR (Módulo: Gestión de Bases de
Datos)
--
================================================================================
DROP DATABASE IF EXISTS practica_dml;
CREATE DATABASE practica_dml CHARACTER SET utf8mb4 COLLATE
utf8mb4_spanish_ci;
USE practica_dml;
-- 1. Creación de la tabla DEPARTAMENTO
-- Demuestra el uso de tipos numéricos estándar e incrementos de
tamaño en nombres
CREATE TABLE departamento (
codigo INT PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,
presupuesto DECIMAL(12,2) DEFAULT 0.00 -- Diferenciación de escala en
punto fijo
) ENGINE=InnoDB;
-- 2. Creación de la tabla OFICINA
-- Incorpora SMALLINT para optimizar almacenamiento en bytes
físicos
CREATE TABLE oficina (
codigo_oficina VARCHAR(10) PRIMARY KEY,
ciudad VARCHAR(50) NOT NULL,
telefono VARCHAR(20) NOT NULL,
superficie SMALLINT UNSIGNED NOT NULL CHECK (superficie > 0) --
Espacio físico en metros cuadrados
) ENGINE=InnoDB;
-- 3. Creación de la tabla EMPLEADO
-- Integra fechas de contratación (DATE), horas de entrada (TIME) y
estados lógicos (BOOLEAN)
CREATE TABLE empleado (
codigo_empleado INT PRIMARY KEY,
nombre VARCHAR(50) NOT NULL,
apellido1 VARCHAR(50) NOT NULL,
email VARCHAR(120) UNIQUE NOT NULL,
salario DECIMAL(10,2) NOT NULL,
comision DECIMAL(10,2) DEFAULT NULL,
codigo_oficina VARCHAR(10),
codigo_departamento INT,
```

```
codigo_jefe INT,
fecha_contratacion DATE NOT NULL,
hora_entrada TIME DEFAULT '08:00:00', -- Ejemplo de tipo de tiempo
(hora del turno)
activo BOOLEAN DEFAULT TRUE, -- Atributo booleano de control de
accesos
FOREIGN KEY (codigo_oficina) REFERENCES oficina(codigo_oficina) ON
UPDATE CASCADE ON DELETE SET NULL,
FOREIGN KEY (codigo_departamento) REFERENCES departamento(codigo) ON
UPDATE CASCADE ON DELETE SET NULL,
FOREIGN KEY (codigo_jefe) REFERENCES empleado(codigo_empleado) ON
UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB;
-- 4. Creación de la tabla CLIENTE
-- Combina lógica de fidelización VIP (BOOLEAN), precisión decimal
ampliada y marcas temporales (DATETIME)
CREATE TABLE cliente (
codigo_cliente INT PRIMARY KEY,
nombre_cliente VARCHAR(100) NOT NULL,
codigo_empleado_rep_ventas INT,
pais VARCHAR(50) DEFAULT 'España',
es_premium BOOLEAN DEFAULT FALSE, -- Clasificación booleana
limite_credito DECIMAL(12,2) DEFAULT 1000.00, -- Capacidad de
solvencia fiscal
fecha_alta DATETIME DEFAULT CURRENT_TIMESTAMP, -- Sello temporal
completo de alta
FOREIGN KEY (codigo_empleado_rep_ventas) REFERENCES
empleado(codigo_empleado) ON UPDATE CASCADE ON DELETE SET NULL
) ENGINE=InnoDB;
-- 5. Creación de la tabla FABRICANTE
-- Contiene un campo TEXT para almacenar bloques de documentación
técnica de gran tamaño
CREATE TABLE fabricante (
codigo INT PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
pais VARCHAR(50) NOT NULL,
descripcion TEXT DEFAULT NULL -- Almacena grandes volúmenes de texto
libre
) ENGINE=InnoDB;
-- 6. Creación de la tabla PRODUCTO
CREATE TABLE producto (
codigo_producto VARCHAR(15) PRIMARY KEY,
nombre VARCHAR(100) NOT NULL,
precio_venta DECIMAL(10,2) NOT NULL CHECK (precio_venta >=
0.00),
gama VARCHAR(50) NOT NULL,
```

```
codigo_fabricante INT NOT NULL,
stock_seguridad INT DEFAULT 10,
FOREIGN KEY (codigo_fabricante) REFERENCES fabricante(codigo) ON
UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
-- 7. Creación de la tabla PEDIDO
-- Incluye timestamps de auditoría de transacciones SysAdmin
CREATE TABLE pedido (
codigo_pedido INT PRIMARY KEY,
codigo_cliente INT NOT NULL,
fecha_pedido DATE NOT NULL,
hora_pedido TIME NOT NULL,
fecha_esperada DATE NOT NULL,
fecha_entrega DATETIME DEFAULT NULL, -- Puede demorarse o contener
horas de descarga
comentarios TEXT DEFAULT NULL,
ultima_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP, -- Auditoría SysAdmin
FOREIGN KEY (codigo_cliente) REFERENCES cliente(codigo_cliente) ON
UPDATE CASCADE ON DELETE CASCADE
) ENGINE=InnoDB;
-- 8. Creación de la tabla GRADO_NOMINA
CREATE TABLE grado_nomina (
grado_salarial INT PRIMARY KEY,
salario_minimo DECIMAL(10,2) NOT NULL,
salario_maximo DECIMAL(10,2) NOT NULL
) ENGINE=InnoDB;
--
================================================================================
-- INSERCIÓN DE DATOS DE EJEMPLO (DML - SEEDING ENRIQUECIDO)
--
================================================================================
-- Inserciones en DEPARTAMENTO
INSERT INTO departamento (codigo, nombre, presupuesto) VALUES
(10, 'Dirección General', 350000.00),
(20, 'Ventas y Expansión', 180000.00),
(30, 'Soporte de Sistemas y DevOps', 250000.00),
(40, 'Recursos Humanos y RSE', 90000.00),
(50, 'I+D+i Tecnológico', 150000.00), -- Nuevos departamentos para
cruces avanzados
(60, 'Marketing y Comunicación', 0.00);
-- Inserciones en OFICINA
INSERT INTO oficina (codigo_oficina, ciudad, telefono, superficie)
VALUES
('OFI-MAD', 'Madrid', '+34 912 345 678', 1200),
('OFI-BAR', 'Barcelona', '+34 931 234 567', 850),
```

```
('OFI-SEV', 'Sevilla', '+34 954 123 456', 500),
('OFI-VAL', 'Valencia', '+34 963 987 654', 420),
('OFI-BIL', 'Bilbao', '+34 944 555 666', 310);
-- Inserciones en EMPLEADO (Cumpliendo dependencias jerárquicas de
jefes)
-- Directores de nivel 1 (jefe NULL)
INSERT INTO empleado (codigo_empleado, nombre, apellido1, email,
salario, comision, codigo_oficina, codigo_departamento, codigo_jefe,
fecha_contratacion, hora_entrada, activo) VALUES
(1, 'Alberto', 'Gómez', 'a.gomez@empresa.com', 4200.00, NULL,
'OFI-MAD', 10, NULL, '2020-01-15', '07:30:00', TRUE);
-- Mandos intermedios y coordinadores (jefe: 1)
INSERT INTO empleado (codigo_empleado, nombre, apellido1, email,
salario, comision, codigo_oficina, codigo_departamento, codigo_jefe,
fecha_contratacion, hora_entrada, activo) VALUES
(2, 'Beatriz', 'Ruiz', 'b.ruiz@empresa.com', 2800.00, 500.00,
'OFI-BAR', 20, 1, '2021-03-10', '08:00:00', TRUE),
(3, 'Carlos', 'López', 'c.lopez@empresa.com', 2400.00, NULL,
'OFI-MAD', 30, 1, '2021-06-01', '08:00:00', TRUE),
(4, 'Sonia', 'Benítez', 's.benitez@empresa.com', 1900.00, 150.00,
'OFI-SEV', 20, 1, '2022-09-18', '08:30:00', TRUE);
-- Técnicos y personal operativo
INSERT INTO empleado (codigo_empleado, nombre, apellido1, email,
salario, comision, codigo_oficina, codigo_departamento, codigo_jefe,
fecha_contratacion, hora_entrada, activo) VALUES
(5, 'José', 'Sanz', 'j@empresa.com', 1500.00, NULL, 'OFI-MAD', 30, 3,
'2023-11-20', '08:00:00', TRUE),
(6, 'Diana', 'Mendoza', 'd.mendoza@empresa.com', 1450.00, NULL,
'OFI-BAR', 30, 3, '2024-02-01', '08:15:00', TRUE),
(7, 'Felipe', 'Torres', 'f.torres@empresa.com', 1200.00, 50.00,
'OFI-SEV', 20, 4, '2024-05-15', '09:00:00', FALSE), -- Empleado
inactivo
(8, 'Elena', 'Guerrero', 'e.guerrero@empresa.com', 1700.00, NULL,
'OFI-VAL', 40, 1, '2022-10-10', '08:30:00', TRUE);
-- Inserciones en CLIENTE (Con combinaciones booleanas y límites de
créditos diversos)
INSERT INTO cliente (codigo_cliente, nombre_cliente,
codigo_empleado_rep_ventas, pais, es_premium, limite_credito,
fecha_alta) VALUES
(101, 'Jardines de Almería S.L.', 2, 'España', TRUE, 50000.00,
'2022-01-10 10:30:00'),
(102, 'Viveros del Cantábrico', 4, 'España', FALSE, 12000.00,
'2023-04-15 11:00:00'),
(103, 'Suministros Globales Portugal', 2, 'Portugal', TRUE, 75000.00,
'2023-08-20 09:15:00'),
(104, 'Cliente Autónomo Huerto', NULL, 'España', FALSE, 500.00,
```

```
'2024-11-01 16:45:00'),
(105, 'Boutique Floral París', 4, 'Francia', TRUE, 120000.00,
'2025-02-18 12:00:00'),
(106, 'AgroInsumos del Sur S.A.', 2, 'España', TRUE, 45000.00,
'2025-05-30 08:30:00'),
(107, 'Horticultura Silvestre', NULL, 'Chile', FALSE, 3000.00,
'2026-01-15 14:22:00'),
(108, 'Jardines de Toscana', 8, 'Italia', TRUE, 90000.00, '2026-03-10
10:00:00');
-- Inserciones en FABRICANTE (Incluso con bloques descriptivos
extensos TEXT)
INSERT INTO fabricante (codigo, nombre, pais, descripcion) VALUES
(1, 'Bellota', 'España', 'Fabricante líder en herramientas de mano de
forjado de alta resistencia y durabilidad para agricultura.'),
(2, 'Gardena', 'Alemania', 'Líder europeo en herramientas de riego
inteligente, robótica de jardín y accesorios ergonómicos.'),
(3, 'Husqvarna', 'Suecia', 'Maquinaria pesada forestal y de
jardinería profesional, incluyendo motosierras, sopladoras e hilos de
corte.'),
(4, 'Xiaomi', 'China', 'Marca global de tecnología inteligente,
incluyendo sensores de humedad de tierra IoT y control inteligente de
invernaderos.'),
(5, 'Sanz', 'España', 'Distribuidor tradicional de abonos minerales
de lenta liberación y fitosanitarios domésticos ecológicos.'),
(6, 'Fiskars', 'Finlandia', 'Herramientas de corte de precisión,
tijeras de podar de alta gama y hachas ligeras con tecnología de
absorción.');
-- Inserciones en PRODUCTO (Gamas: Herramientas, Frutales,
Ornamentales)
INSERT INTO producto (codigo_producto, nombre, precio_venta, gama,
codigo_fabricante, stock_seguridad) VALUES
('PROD-H01', 'Maza de goma pesada', 125.50, 'Herramientas', 1,
15),
('PROD-H02', 'Tijera de podar ergonómica', 45.00, 'Herramientas', 2,
30),
('PROD-H03', 'Hacha de mano Fiskars X7', 55.00, 'Herramientas', 6,
12),
('PROD-F01', 'Manzano de flor injertado', 85.00, 'Frutales', 1,
5),
('PROD-F02', 'Cerezo japonés', 150.00, 'Frutales', 2, 8),
('PROD-F03', 'Limonero de cuatro estaciones', 110.00, 'Frutales', 5,
4),
('PROD-O01', 'Rosa silvestre', 25.00, 'Ornamentales', 1, 50),
('PROD-O02', 'Maza de madera de arce', 75.00, 'Ornamentales', 3,
20),
('PROD-O03', 'Orquídea Phalaenopsis Premium', 35.00, 'Ornamentales',
```

```
5, 25),
('PROD-004', 'Sensor de humedad IoT Bluetooth', 19.99,
'Ornamentales', 4, 100);
-- Inserciones en PEDIDO (Combina fecha, hora y fecha
esperada/entrega precisa)
INSERT INTO pedido (codigo_pedido, codigo_cliente, fecha_pedido,
hora_pedido, fecha_esperada, fecha_entrega, comentarios) VALUES
(501, 101, '2026-05-15', '09:30:00', '2026-06-01', '2026-05-31
14:30:00', 'Cliente solicita descarga en muelle posterior.'),
(502, 102, '2026-06-01', '11:15:00', '2026-06-15', '2026-06-20
18:00:00', 'Retraso logístico ajeno a fábrica. Compensar portes.'),
(503, 101, '2026-06-18', '08:00:00', '2026-07-01', NULL, 'Pendiente
de confirmación de aduanas.'),
(504, 105, '2026-06-10', '16:00:00', '2026-06-25', '2026-06-24
10:15:00', 'Pedido internacional prioritario.'),
(505, 103, '2026-06-12', '12:45:00', '2026-06-28', NULL, 'En tránsito
terrestre internacional.'),
(506, 106, '2026-06-14', '10:00:00', '2026-06-30', '2026-06-29
09:00:00', 'Entregado en palets termo-sellados.'),
(507, 108, '2026-06-15', '17:30:00', '2026-07-05', NULL, 'Envío
especial consolidado.'),
(508, 101, '2026-06-16', '14:20:00', '2026-07-01', NULL, 'Segundo
pedido recurrente del mes.');
-- Inserciones en GRADO_NOMINA
INSERT INTO grado_nomina (grado_salarial, salario_minimo,
salario_maximo) VALUES
(1, 1000.00, 1999.99),
(2, 2000.00, 2999.99),
(3, 3000.00, 4199.99),
(4, 4200.00, 6000.00); -- Ampliado a grado 4 para cubrir salarios de
directores principales
```

```
--
================================================================================
-- BASE DE DATOS DE PRÁCTICAS DML - UNIDAD 5 (ESQUEMA ENRIQUECIDO
PARA ASIR)
-- Diseñado para el ciclo de ASIR (Módulo: Gestión de Bases de
Datos)
--
================================================================================
DROP DATABASE IF EXISTS practica_dml;
CREATE DATABASE practica_dml CHARACTER SET utf8mb4 COLLATE
utf8mb4_spanish_ci;
USE practica_dml;
-- 1. Creación de la tabla DEPARTAMENTO
```

```
CREATE TABLE departamento (
codigo INT PRIMARY KEY,
nombre VARCHAR(80) NOT NULL,
presupuesto DECIMAL(12,2) DEFAULT 0.00
) ENGINE=InnoDB;
-- [Resto del script truncado por brevedad, siguiendo el patrón de
los ejemplos anteriores]
```